ITFC: A Blockchain System with Incentivized Transaction Forwarding in Cloud Network

Jiarui Zhang, Yaodong Huang, Yiming Zeng, Xiaojun Shang, and Yuanyuan Yang, *Fellow, IEEE*

Abstract—Blockchain, originally introduced for cryptocurrencies, provides a secure and decentralized platform for processing and storing transactions among distributed cloud nodes. However, traditional blockchain systems require broadcasting each transaction to all nodes, incurring high computational and communication overhead. In cloud networks, this problem is exacerbated because forwarding transactions consumes limited bandwidth and computing resources, which can discourage nodes from participating honestly. In addition, some nodes may refuse to forward transactions, undermining network consensus. In this paper, we propose Incentive Transaction Forwarding in Cloud networks (ITFC), which rewards cloud nodes with a share of transaction fees for forwarding transactions. We design a dynamic topology update mechanism to enable accurate incentive computation and develop a specified blockchain structure to support fair revenue distribution. We analyze the security of ITFC, proving its robustness against common adversarial strategies and showing that it prevents unfair gains. Extensive simulations demonstrate that ITFC achieves fair incentive allocation for relay nodes while mitigating adversarial attacks.

Index Terms—Blockchain, Cloud network, Incentive allocation, Transaction forwarding, Security

1 Introduction

With the advent of high-speed, low-latency networking technologies such as 5G and Wi-Fi 6 [1], a rapidly growing number of devices—including smartphones, IoT devices, and connected vehicles—are transforming daily life through the exchange of massive volumes of data. These devices are increasingly equipped with sufficient sensing, computing, and storage capabilities. Leveraging these capabilities, users can generate and provide data and services to others, thereby driving the demand for efficient and secure micropayment systems.

Blockchain technology [2], originally introduced as a secure and decentralized system for trusted transactions, has been widely adopted in cryptocurrencies. It provides a tamper-resistant ledger for recording transaction histories, participant credits, and related information. However, the rapid growth of data has significantly increased the size of modern blockchains. Unlike the original design, which targeted decentralized independent users, today's blockchain systems have become so resource-intensive that users with limited devices can no longer easily store or access the entire chain independently. As a result, personal devices increasingly rely on cloud servers with greater computational and storage capacities to access blockchain services. These cloud servers collectively form a blockchain cloud network, which connects ordinary end-users and enables the operation of the overall system.

Broadcasting all transactions is essential for maintaining blockchain consensus. To ensure agreement on the blockchain's state across different cloud nodes, every transaction must be propagated throughout the network. This broadcast process depends on cloud nodes voluntarily forwarding transactions as relays, an inherently altruistic behavior. However, handling massive transaction volumes imposes significant computational and communication overhead, since nodes must forward transactions without receiving compensation. Consequently, some nodes may delay or even refuse to forward transactions due to the lack of incentives [3]. Such delays or losses in transaction propagation threaten both data security and the consensus integrity of the blockchain system.

To further motivate cloud nodes to forward transactions and to compensate for the resources they consume in the process, we introduce incentive allocation, where a portion of transaction fees is distributed as revenue to relay nodes. This design raises several key challenges. First, since forwarding requires both computing and communication resources, it is necessary to accurately measure the contribution of each cloud node during transaction propagation and to allocate revenue proportionally to their efforts. Second, contributions must be evaluated with respect to the cloud network topology, which is inherently dynamic. The allocation mechanism must therefore adjust continuously to reflect changing topologies in real time. Third, distributing revenue among relay nodes introduces potential security risks because adversaries may attempt to manipulate forwarding behavior or exploit network structures to gain excessive rewards. The incentive allocation scheme must be designed with strong security guarantees so that adversaries cannot obtain unfair profits over honest nodes.

- *J. Zhang and Y. Yang are with the Department of Electrical and Computer Engineering, Stony Brook University, Stony Brook, NY 11794, USA. E-mail: {jiarui.zhang.2, yuanyuan.yang}@stonybrook.edu*
- *Y. Huang is with the College of Computer Science and Software Engineering, Shenzhen University, Shenzhen, China. E-mail: yd.huang@szu.edu.cn*
- *Yiming Zeng is with the School of Computing, Binghamton University, Binghamton, NY 13902, USA. Email: yzeng4@binghamton.edu*
- *Xiaojun Shang is with the Department of Computer Science and Engineering, The University of Texas at Arlington, Arlington, TX 76019, USA. Email: xiaojun.shang@uta.edu*

In this paper, we propose Incentivized Transaction Forwarding in Clouds (ITFC), a blockchain framework that enables relay cloud nodes to earn revenue for participating in transaction forwarding. To support the purpose, we introduce a mechanism that dynamically maintains changes in the cloud network topology and design algorithms that fairly distribute transaction fees among relay nodes based on their contributions. We further conduct a security analysis against several common attack strategies and demonstrate that our algorithms prevent adversaries from gaining unfair profits. Finally, through extensive simulations, we show that ITFC achieves fair revenue allocation for relay nodes while maintaining strong resilience against multiple types of attacks.

The main contributions of this paper are as follows.

- We propose a novel blockchain system, ITFC, that focuses on secure and fair incentive allocation for relay nodes in cloud computing blockchain networks.
- We design three algorithms for revenue distribution among cloud nodes that forward transactions. The first algorithm reduces the network topology to decrease the number of links considered when computing incentives. The second algorithm calculates the revenue for each cloud node based on its transaction forwarding contribution. The third algorithm computes the revenue associated with serving user nodes.
- To track and maintain an up-to-date cloud computing network topology for incentive allocation, we develop a mechanism and extend the blockchain block structure to support the application of our algorithms.
- From a security perspective, we analyze ITFC under common attack scenarios and demonstrate that it prevents adversaries from obtaining unfair profits through the revenue-sharing mechanism.
- We perform extensive simulations to evaluate the incentive allocation and security performance. The results show that ITFC fairly distributes revenue to honest nodes and effectively resists multiple types of attacks aimed at gaining unfair profits.

The rest of this paper is organized as follows. Section 2 reviews related work. In Section 3, we formulate the incentive allocation problem and present the overall system model. Section 4 introduces methods for maintaining the topology of cloud blockchain networks. In Section 5, we present our incentive allocation algorithms. Section 6 describes the ITFC blockchain structure. Section 7 analyzes potential attacks and the security of the system. In Section 8, we evaluate the incentive allocation and assess the system's performance against various malicious attacks. Finally, Section 9 concludes the paper.

2 Related Work

Blockchain technology was first introduced in 2008 by Satoshi Nakamoto [2] and has been one of the most popular Peer-to-Peer (P2P) structures. A blockchain consists of blocks, each of which has a unique authentication method to confirm a unique order and integrity. Since the blockchain is often built for untrusted parties, the fairness of the blockchain is important and has been well studied in multiple aspects. Most of the existing studies on the fairness of blockchain mainly focus on mining and rewarding mechanisms. Eyal and Sirer [4] proved that users can gain more profits than deserved by strategic mining in Bitcoin. Pass and Shi [5] proposed their Fruitchain design to improve the fairness of mining. Amoussou-Guenou et al. [6] fully defined the fairness in Tendermint-core blockchains and propose a modification of the Tendermint rewarding to match their proposed fairness.

In P2P environments, participants are consumers who require services from others, and they are also providers who provide services to others in the network. To keep a balance between consumers and providers in P2P environments, incentive allocation algorithms are developed. The incentive allocation algorithms often encourage information exchanges, service supports, and collaborative works. Most of the traditional incentive allocation algorithms are based on economics and game theory. Feldman et al. [7] proposed several incentive techniques in the P2P systems to optimize levels of cooperation. Yan et al. [8] gave an autonomous compensation game model to encourage data exchange in a spatial restricted P2P system. In dynamic and distributed P2P environments, He et al. proposed an incentive mechanism based on the blockchain [9]. Their mechanism rewards intermediate nodes on the path from the sender to the receiver and imports the blockchain system to resist attacks. However, the incentive allocation algorithms applicable in P2P environments may not apply to blockchain environments. For instance, Buttyan et al. [10] proposed barter-based schemes such that users can trace the historical records of other users and prevent several attacks, but users are not supposed to have long historical records in blockchain environments. Thus, incentive schemes for blockchain applications need to be more sophisticated for the blockchain. Research which are based on reputation [11] [12] deploy reputation systems to measure contributions and incentives, but they need centralized certificate authorities to avoid whitewashing attacks [13] or Sybil attacks [14].

As an anonymous decentralized system, the participants are supposed to be anonymous in the blockchain network. However, interactions between blockchain nodes construct a topology over the blockchain network. Although blockchain network may be huge, blockchain network topology detection is applicable. Franzoni et al. [15] argue that an open topology is beneficial, and propose a protocol to infer connections among reachable nodes of the Bitcoin network. Essaid et al. [16] analyze the Bitcoin network properties, community structure by detecting the Bitcoin network topology. Zhao et al. [17] propose an algorithm to detect blockchain topology and prove the accuracy of the result. Their work demonstrates the feasibility of a node in a network detecting the topology of the entire blockchain network. These works focus on network topology detection from the perspective of a node in the blockchain network.

Incentive allocations in the blockchain still have a lot of room for improvement. The Bitcoin Lightning Network [18] encourages users to conduct off-chain transactions through channels, and the transaction senders pay fees to the intermediate nodes that establish the channels. As a result, most

transactions become off-chain transactions, thereby saving on-chain space and increasing transaction throughput. The current research focus of the Bitcoin Lightning Network is to improve the transmission efficiency [19] and maximize the profit of a single node [20], [21]. Sharding [22] is another technology to improve the throughput of blockchain. It divides the network into smaller committees, each of which can work independently. The current incentive scheme of the sharding is to improve the reputation of the nodes that perform well, and the nodes with higher reputations have higher probabilities to be selected as leaders of the committee [23], [24]. The reputation method is mainly to improve the security of the sharding by optimizing the selection of the committee, rather than an incentive allocation strategy. Meanwhile, blockchain technologies provide a kind of new possibilities to resolve the security and privacy problems in the Internet of Things (IoT) [25] and edge computing [26] environments. Ding et al. [27] studied a specialized incentive mechanism for the blockchain to attract IoT devices to purchase computational power from edge servers to participate in the mining process. Huang et al. [28] proposed a hybrid system for edge computing environments and analyzed the incentive allocation algorithm in the system. Most of these works are based on their specific application scenarios, thus generalized algorithms still have room for development.

There are some works on the incentive for relay nodes in blockchain systems. The techniques and environments used in these works are diverse. Babaioff et al. [3] addressed that there is no motivation for nodes to broadcast transactions, and proposed a reward scheme that motivates nodes to propagate transactions. Their scheme is based on the assumption that the network is modeled as a forest of d-ary directed trees, each of them of height H. Abraham et al. [29] designed a signed propagation chain solution for their proposed blockchain called Solidus to motivate information propagation. The signed propagation chain requires support from the committee of Solidus. Moreover, the chain requires all transactions to be continuously encrypted according to the propagation path. This results in completely different versions of the same transaction processed by all nodes, thus additional overhead may be required during decryption and synchronization. Ersoy et al. [30], [31] found a formula that distributes the transaction fee among propagating nodes. Their formula includes a critical parameter related to the network topology, and the choice of this parameter is an open problem. Li et al. [32] proposed CreditCoin to incentivize vehicles to forward announcements in a blockchain-based network. Because their solution relies on the trusted authority and a cloud server, it is not centralized and cannot be applied in general networks. Wang et al. [33] proposed a solution to a UTXO-based blockchain network. Because the solution assumes that all nodes in the network only propagate transactions to a certain miner, their solution cannot be applied when the miner is uncertain. In addition, their solution requires a large number of additional transactions for each transaction to distribute revenue to all nodes on the path.

3 Problem Background

In this section, we discuss the background of incentive allocations for cloud relay nodes. We further propose corresponding formulations and models to solve the problem. In our proposed fair incentive allocation, cloud relay nodes are given incentives to compensate costs and encourage transaction forwarding.

3.1 Cloud Blockchain Network

With the growing scale of blockchain networks, the large volume of on-chain data and the high rate of transaction forwarding make it increasingly difficult for individual users with limited computing power to perform storage, forwarding, and mining operations. For reference, the Bitcoin blockchain had reached approximately 653 gigabytes by July 2025, growing at nearly one gigabyte every few days [34]. Mining power in Bitcoin is also highly concentrated: the largest mining pool controls 27.4% of the total mining power, and the top ten pools together account for 91.54% of the total mining power [35].

The heterogeneous distribution of resources among network participants necessitates a functional classification of nodes. We define two types: cloud nodes and user nodes. Cloud nodes are entities with substantial storage, computational, and network capacity, enabling them to execute critical functions such as mining, transaction relay, and maintaining the full blockchain ledger. Conversely, user nodes are typically lightweight clients, such as digital wallets, that originate transactions and depend upon the infrastructure provided by cloud nodes, compensating them via transaction fees. In our subsequent analysis, we posit that the set of cloud nodes is synonymous with the set of relay nodes within the network.

The cloud blockchain network is denoted as a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ indicates the set of cloud nodes and $\mathcal{E}$ indicates the set of links between different cloud nodes. A set of user nodes $\mathcal{U}$ includes all user nodes, and each user node requires services from a cloud node. For each cloud node $V \in \mathcal{V}$, a set $\mathcal{U}_V$ includes all user nodes that require service from V.

The cloud nodes need to be registered on the network to let the user nodes know that they provide services. The registration of cloud nodes is feasible in our model because the number of cloud nodes is much smaller than the user nodes (22 thousand versus 1 million in Bitcoin).

In a real blockchain network, a real user can require services from multiple cloud servers. For simplicity, we let the real user implement this case by creating multiple user nodes in the model, and each user node requires service from different cloud nodes.

3.2 Background of Incentive Allocation for Cloud Nodes

In traditional blockchain systems, block generators receive revenue for processing and authorizing transactions. The revenue includes two parts. First, to encourage nodes to maintain the blockchain, the blockchain system gives revenue to the block generators. Second, to encourage the block generator to authorize a transaction, the transaction

proposer associates a transaction fee with the transaction. The block generator receives the fee after authorizing the transaction.

Whether providing cloud servers for mining or creating cloud mining pools, cloud nodes contribute to the blockchain system by taking on the responsibility of relaying the blockchain. Therefore, in addition to block generators, the forwarding behavior of relay cloud nodes also needs to be incentivized. Since each node in the network can be a potential new block generator of the blockchain, transactions must be broadcast to all nodes in the blockchain network. This allows a node to validate as many transactions as possible when it successfully mines a new block, that is, finding the solution to the Proof-of-Work (PoW) consensus. Otherwise, the block generator must collect and verify transactions from the entire network once it successfully mines a new block, which might significantly slow down the consensus process and increase the risk of forks. To effectively and reliably broadcast transactions, each cloud node needs to forward transactions to its neighbors. Thus, cloud nodes play a critical role in transaction broadcasting and blockchain consensus.

However, nodes may be motivated to refuse forwarding transactions for their own benefit. Babaioff et al. [3] pointed out that a node may hold a transaction to prevent others from knowing it, thus increasing the probability of obtaining the corresponding transaction fee in future blocks. Meanwhile, there is no penalty for those nodes that do not forward transactions in the current blockchain systems. For example, in the current Bitcoin P2P network [36], the protocol requires a node to forward at least one transaction through a link in 30 minutes to keep the link connected. Nodes can only forward a small portion of transactions, thereby reducing bandwidth consumption while maintaining links with peers. This results in low forwarding efficiency of the network.

For these reasons, we allocate revenue to the cloud nodes and incentivize them to forward transactions, i.e., Incentivized Transaction Forwarding for Clouds (ITFC). Transaction proposers of traditional blockchain systems like Bitcoin attach transaction fees for miners. When applying ITFC, the transaction proposer needs to pay for broadcasting the transaction, therefore the transaction fee is also allocated to cloud nodes.

4 Cloud Node Topology

This section presents our framework for the blockchain cloud network topology. We first contextualize the problem by discussing the necessity of such a topology and the inherent complexities in its management. Subsequently, we detail our methodology for maintaining the topological structure under the conditions of our model.

4.1 Background of Cloud Network Topology

In blockchain networks, tracking the topology is challenging. First, the blockchain network allows nodes to join and leave the network. Nodes are challenging to track and synchronize with the dynamically changing network topology. Second, the number of nodes can be very high. The cost of tracking the topology of a huge network could be high.

Recording the cloud blockchain network topology on-chain is feasible for two main reasons. First, while the number of user nodes in a blockchain can be very large, the number of cloud nodes is relatively limited. For example, in the Bitcoin network—the largest blockchain network—there were approximately 1 million active addresses [37] and around 22000 full nodes [38] by July 2025. In this context, Bitcoin addresses represent user nodes, and full nodes correspond to cloud nodes, making it practical to maintain the cloud node network topology. Second, each cloud node has a limited number of connections [39], [40], which constrains the total number of links between cloud nodes and allows the topology to be effectively recorded and maintained.

The incentive allocation needs to consider the contributions of cloud nodes. Before we discuss the definition of cloud node contribution in forwarding transactions, it is obvious that not all cloud nodes contribute equally consistently. For example, in a star graph[1], the internal node contributes most to the graph because any communication between two different nodes should go through the internal node. The cloud network topology is critical when considering the contribution of the cloud node in forwarding transactions.

4.2 Tracking Topology

In a dynamically changing blockchain network, nodes and links are constantly evolving. Tracking the network topology therefore requires monitoring changes in nodes and links. For simplicity, unless otherwise specified, the term "node" refers to a cloud node when discussing network topology.

4.3 Change of Nodes

The joining and leaving of nodes are common phenomena in blockchain networks. Whenever a new node joins or an existing node leaves, the network topology changes accordingly.

4.3.1 Nodes Join the Network

As described in Section 3, cloud nodes are required to register to join the network. A node's registration signals its entry into the network. When a new node wishes to join, it broadcasts a cloud node registration message. Upon generation of a new block, the block generator records all valid registration messages on the blockchain. Once the registration message is validated, the newly joined node can begin functioning as a cloud node within the blockchain network.

4.3.2 Nodes Leave the Network

In general, capable nodes provide long-term and stable services within the network. However, nodes may temporarily go offline due to factors such as network fluctuations or system failures. Unlike registration, nodes do not send specific messages to indicate that they are leaving the network, so the blockchain does not record node departures.

1. A star graph is a bipartite graph that one internal node has one edge to every other nodes.

When a cloud node leaves the network, it impacts both other cloud nodes and user nodes. Cloud nodes that maintain links to the departed node will disconnect these links according to their offline tolerance, which we discuss in the next subsection. User nodes connected to the departed node will no longer be able to propose transactions or retrieve blockchain information from it, and will instead interact with alternative cloud nodes for services.

4.4 Change of Links

A link connects two distinct nodes in the blockchain network. A change in links occurs when two nodes either establish a new connection or remove an existing one. We refer to these changes as connecting events and disconnecting events, respectively.

4.4.1 Connecting events

Two nodes establish a connection through a new link when they can directly communicate with each other. Connecting events occur when new nodes join the network or when existing nodes attempt to improve their connectivity. The process of connecting events is described as follows:

- Two nodes set up a new link for forwarding transactions.
- Both nodes broadcast a message to announce that the link is set up. Each message is signed by the proposer.
- The new block generator validates a link by validating the signed messages from both proposers. The new link messages are recorded to the new block.
- The links recorded to the blockchain serve for forwarding transactions.

Similar to transaction proposers paying transaction fees to block generators, nodes that generate new links also pay fees to block generators. Two nodes connected through a link need to pay fees for establishing the link. It serves two purposes. First, it encourages nodes to maintain existing links instead of frequently replacing links to enhance the stability of the network. Second, it can prevent malicious nodes from filling a large number of connecting events in the block to achieve the effect of denial-of-service (DoS) attacks. Therefore, a reasonable payment method must not only enable honest nodes to gain benefits while maintaining links but also prevent the network from being attacked by DoS attacks.

4.4.2 Disconnecting events

A node disconnects from an existing link when it can no longer communicate with the other node. Disconnecting events may occur due to node departures, network bandwidth limitations, or power outages. Typically, the node causing the disconnection does not broadcast a message to indicate a proactive departure. Therefore, unlike connecting events that require confirmation from both endpoints, a link can become invalid if any endpoint reports the loss of the connection. A node can broadcast a message indicating that it has terminated a link with another node.

Note that links can be declared invalid unilaterally. Disabling links can help save network bandwidth and reduce power consumption, so nodes may proactively shut down some connections. However, this may also reduce the overall network connectivity. It is therefore essential to ensure that nodes cannot gain additional profits by disabling any active links, preventing malicious disconnecting events. We will demonstrate in Section 5 that nodes cannot increase their revenue by disabling links.

4.5 User Node Activity

For each cloud node $i \in \mathcal{V}$, a set $\mathcal{U}_i$ includes all user nodes that require service from i. Theoretically, a cloud node i with a set $\mathcal{U}_i$ with more user nodes contributes more to the network. However,

We track the topology of the graph composed of cloud nodes. However, tracking $\mathcal{U}_i$ is harder. Although maintaining the topology of cloud nodes is acceptable, $\mathcal{U}_i$ is hard to maintain. First, the numerous number of user nodes makes large $\mathcal{U}_i$ sets for every cloud node. Second, user nodes are not stable like cloud nodes, and they may change the cloud nodes for services frequently.

To resolve this issue, we propose the concept of the user node activity. The later a user node's latest transaction is, the more actively it is. This criterion is based on the fact that most nodes do not participate in the blockchain continuously. A study in Ethereum [39] shows that most nodes do not transfer funds frequently. We consider that most nodes that do not transfer funds frequently are inactive. Compared with such inactive nodes, nodes that frequently participate in transactions are more actively involved in transaction forwarding. Therefore, the user node activity k is defined by the index of the latest block that includes a transaction where the user node is the payer or payee. For a blockchain $\mathcal{B} = \{B_1, B_2, ..., B_n\}$ with the latest block B_n, a user node with activity k indicates that the latest transaction related to the user node occurs in the block B_k.

Therefore, we only consider active user nodes in $\mathcal{U}_i$. While computing the related metrics or values, we consider only user nodes in $\mathcal{U}_i$ whose activity k satisfies $k > n - H$, where n is the current block height and H is a constant representing the maximum allowed activity gap. For simplicity, we redefine $\mathcal{U}_i$ as a set that only includes active user nodes in the following discussions.

5 Incentive Allocation Algorithm

In this section, we propose a novel incentive allocation algorithm to distribute revenue among cloud nodes. We first explain the relationship between revenue from mining and transaction forwarding. Then, we present the objectives of the allocation, describe the algorithm in detail, and provide comprehensive analyses.

5.1 Revenue for Mining and Forwarding

User nodes that propose transactions attach transaction fees for forwarding and validating the proposed transactions. There exists a standard transaction fee, which is low but not trivial, to set a threshold that is acceptable to all demanders but prevents attackers from sending a large number of invalid transactions. For a demander that needs to speed up the validation process of a transaction, it can increase the

transaction fee to prioritize the corresponding transaction. The transaction fees paid by the transaction proposers are not forced to be the same or standardized. Therefore, we assume that each transaction attaches a non-trivial transaction fee, and this fee will be sent to the relay cloud nodes and the block generator that validates the transaction.

After successful mining, the miner will receive revenue as the mining reward. Meanwhile, cloud nodes are incentivized to forward transactions in the network. However, while applying the incentive allocation for cloud nodes, the cost-to-income ratio of mining and forwarding should be considered. This ratio determines the strategy of cloud nodes for investing resources in mining and forwarding. When improving the connectivity in the network provides more revenue to cloud nodes, they may stop working on mining. In this case, cloud nodes are not motivated to generate new blocks, and the maintenance of the blockchain is compromised.

To avoid this threat, with the same resource cost, the unit revenue of successful mining needs to be no lower than the unit revenue of successful forwarding. The source of the mining revenue includes the transaction fees and the system revenue for new blocks, while the source of the forwarding revenue only includes the transaction fees. If the transaction fees for relaying are less than half of the total transaction fees, the mining revenue will be higher than the forwarding revenue. The cloud nodes will continue to mine because the mining revenue from transaction fees is no less than the forwarding revenue shared by all cloud nodes.

5.2 Goals of the Incentive Allocation Algorithm

We summarize the goals of the incentive allocation algorithm as follows. First, the algorithm should distribute revenue according to the contributions of cloud nodes. This requires a method to measure each node's contribution based on its role in forwarding transactions so that revenue can be assigned proportionally. Second, as discussed in Section 3, cloud nodes can disconnect unilaterally. The algorithm must prevent nodes from increasing their revenue by disconnecting, thereby avoiding malicious disconnecting behaviors. Third, given the large number of cloud nodes in the network, any node could act as a block generator and compute the incentive allocation. Therefore, the algorithm's computational complexity should be low enough to allow timely computation by all nodes. Based on these considerations, we define the primary goals of the incentive allocation algorithm.

- The algorithm allocates revenue by contributions of cloud nodes when relaying transactions.
- Cloud nodes cannot increase their revenue by disconnecting.
- The computational time complexity of the algorithm is acceptable to most cloud nodes.

5.3 Incentive Allocation Algorithm and Analysis

The input of the algorithm consists of a set of transactions $\mathcal{T}$, cloud node set $\mathcal{V}$, and link set $\mathcal{E}$. Transaction t in $\mathcal{T}$ consists of three parts, i.e., $t = (h, s, q, w)$. h denotes the payer of transaction t which starts to broadcast the transaction to the blockchain network. s is the cloud node that provides forwarding service to h. q denotes the payee of transaction t. w denotes the transaction fee. This transaction is signed by h for security.

We discuss the transaction broadcast process. When user node h broadcasts a transaction T to the network, the transaction is sent to s, then s forwards it to all cloud nodes who directly connect to s. Each cloud node will forward the transaction after it receives the transaction. Assume that cloud node i sends cloud node j a transaction message through the link (i, j). If j does not receive this message before, it will record this transaction message and forward it to other nodes who connect to it. Otherwise, the cloud node will not forward this transaction to avoid redundant forwarding processes. Thus, a node at most forwards a transaction once. Each node will forward the transaction if it is the first time to receive the transaction because multiple forwarding processes from the same node cannot send the transaction to more nodes. The forwarding process gives us a transparent framework of broadcast contributions. This means that only if a node receives a transaction via one of the shortest paths from s, it may forward it, since only transactions arriving along a shortest path are possible to be the first instance the node has seen the transaction. Theoretically, only links that belong to the shortest paths are sufficient to forward the transaction to all nodes in the shortest time. A transaction forwarding process over one of these links is called a **sufficient forwarding**.

According to the transaction broadcast process, we make a reduction of the link set $\mathcal{E}$. Since the shortest paths contribute to all of the sufficient forwarding processes, the framework of broadcast contribution is composed of the shortest paths. This fact allows us to simplify the calculation of the incentive allocation through the reduction of the graph. Thus, we propose a reduction that ignores links that are not a part of the shortest paths from s when computing the incentive allocation of the transaction. This reduction keeps the broadcast process efficient because a cloud node receives the transaction from one of the shortest paths from s. This reduction also removes links that are less involved in the broadcast process of the transaction. We propose Algorithm 1 to apply the reduction to the link set.

TABLE 1
Notations used in Algorithms

$\mathcal{E}'$	Reduced edge set after applying reduction algorithm
d_i	The length of the shortest path from s to cloud node i. Cloud Node i is located at level d_i
p_i	The outdegree of i in $(\mathcal{V}, \mathcal{E}')$
r_{d_i}	The ratio of level d_i's revenue to level $M-1$'s revenue
c_{d_i}	The total number of cloud nodes at level d_i
g_{d_i}	The total number of outdegrees of all cloud nodes at level d_i
M	The maximum value of d_i
S	The total amount of r_{d_i} for all possible d_i
a_i	The number of revenue to i
a'_i	The number of revenue to i for serving user nodes
$\mathcal{D}$	The set of d_i
(i, j)	A directed link between i and j

Algorithm 1 retains the links that belong to any shortest path from s to other cloud nodes. Lines 2 to 11 describe

Algorithm 1 Graph Reduction Algorithm

Input: $\mathcal{G} = (\mathcal{V}, \mathcal{E}), t = (s, q, w)$
Output: $\mathcal{E}', \mathcal{D} = \{d_i | i \in \mathcal{V}\}$
1: $d_s \leftarrow 0$
2: Initialize an empty queue Q, then add s to Q
3: **while** Q is not empty **do**
4: Remove the front element of Q and assign it to i
5: **for all** $(i, j) \in \mathcal{E}$ **do**
6: **if** $d_j = \infty$ **then**
7: $d_j \leftarrow d_i + 1$
8: Add j to the back of Q
9: **end if**
10: **end for**
11: **end while**
12: $\mathcal{D} = \{d_i | i \in \mathcal{V}\}$
13: **for all** $(i, j) \in \mathcal{E}$ **do**
14: **if** $d_j = d_i + 1$ **then**
15: $\mathcal{E}' \leftarrow \mathcal{E}' \cup (i, j)$
16: **end if**
17: **end for**

a breadth-first search (BFS) for computing d_i. The time complexity of the algorithm is $O(|\mathcal{V}| + |\mathcal{E}|)$, which matches the complexity of the BFS process.

We use **level** to describe the distance between cloud nodes and s. Cloud node i with distance $d_i = n$ from s is at level n. Any link in $\mathcal{E}'$ connects a cloud node at level n to another node at level $n + 1$. We call s the root since $d_s = 0$. As a special case, when i cannot reach s, $d_i = \infty$.

Next, we propose an algorithm for incentive allocation. Carefully considering the design goals, our algorithm includes mainly three steps. First, the algorithm computes necessary values for the following incentive allocation. Second, the algorithm computes the total revenue of each level according to the network topology to ensure that the revenue of any cloud node will not increase after it disconnects a link. Third, the algorithm counts the number of links between each cloud node and its next level to determine the revenue of each cloud node. The detailed algorithm is presented in Algorithm 2. The notations can be found in Table 1.

Algorithm 2 Incentive Allocation Algorithm

Input: $\mathcal{V}, \mathcal{E}', w, \mathcal{D} = \{d_i | i \in \mathcal{V}\}$
Output: $\mathcal{A} = \{a_i | i \in \mathcal{V}\}$
1: $\forall n \in [0, |\mathcal{V}| - 1]$, initialize $c_n = 0, r_n = 0, g_n = 0$
2: $\forall i \in \mathcal{V}$, initialize $p_i = 0, a_i = 0$
3: **for all** $(i, j) \in \mathcal{E}'$ **do**
4: $p_i \leftarrow p_i + 1$
5: **end for**
6: **for all** $i \in \mathcal{V}$ **do**
7: $c_{d_i} \leftarrow c_{d_i} + 1$
8: $g_{d_i} \leftarrow g_{d_i} + p_i$
9: **end for**
10: Find maximum M, s.t. $c_M \neq 0$
11: $r_{M-1} \leftarrow 1$
12: **for** n from $M - 2$ to 1 **do**
13: $r_n \leftarrow r_{n+1}((c_n - 1)c_{n+1} + 1)/2$
14: $S \leftarrow S + r_n$
15: **end for**
16: **for all** $i \in \mathcal{V}$ **do**
17: $a_i \leftarrow (p_i r_{d_i} w)/(g_{d_i} S)$
18: **end for**

In Algorithm 2, lines 1 to 9 compute the necessary values listed in Table 1 for incentive allocation. Lines 10 to 15 compute the total revenue of each level. $\frac{r_{d_i} w}{S}$ is the actual revenue distributed to level d_i. Lines 16 to 18 compute the revenue for each cloud node. $\frac{p_i}{g_{d_i}}$ is the ratio of the links from node i to the total links from level d_i. We derive the revenue for cloud node i by a_i by $\frac{p_i r_{d_i} w}{g_{d_i} S}$.

We now prove that cloud nodes cannot get more revenue by disconnecting from any other cloud nodes.

Lemma 1. *Given the graph $(\mathcal{V}, \mathcal{E}')$ and a transaction, if cloud node i disconnects a link to another node, d_i will not decrease.*

Proof. Assume that after removing link (i, j) from $\mathcal{E}'$, d_i is decreased to d_i', $d_i' < d_i$. There exists a path $P = (u_1, u_2, ..., u_{d_i'})$ from s to i, satisfying $u_1 = s$, and link (u_x, u_{x+1}) exists in $\mathcal{E}'$ for each integer $x \in [1, d_i' - 1]$. The path P is still available in the graph before removing link (i, j), thus $d_i \leq d_i'$. It contradicts the assumption $d_i' < d_i$.

If cloud node i keeps all its links unchanged, d_i will be unchanged. In addition, i can increase d_i by disconnecting links. For example, when i disconnects all links, $d_i = \infty$. □

Theorem 2. *For a given transaction t, cloud node i cannot get more revenue by disconnecting any link to other nodes, while other links are not changed.*

Proof. From Lemma 1, cloud node i can only keep or increase its level d_i. First, we discuss the case that i keeps its level d_i. Our algorithm computes ratio r_{d_i} for level d_i, then gives $\frac{p_i}{g_{d_i}}$ of total revenue of level d_i to i. r_{d_i} is independent of the number of links. If i disconnects any cloud node at level $d_i - 1$, $\frac{p_i}{g_{d_i}}$ will not change. If i disconnects any cloud node at level $d_i + 1$, $\frac{p_i}{g_{d_i}}$ will decrease. Thus, i cannot get more revenue by disconnecting it from other cloud nodes, while other links are not changed.

Next, we discuss the maximum revenue that i can get when d_i increases. We consider that i moves its level d_i to $d_i + 1$. Every cloud node at level d_i has at most one link to each node at level $d_i + 1$, thus each cloud node has at most c_{d_i+1} links to level $d_i + 1$. There exist at most $(c_{d_i} - 1)c_{d_i+1}$ links from level d_i to level $d_i + 1$ except links of i. Therefore, any cloud node at level d_i receives at least $\frac{1}{(c_{d_i}-1)c_{d_i+1}+1}$ of total revenue at level d_i. When $d_i = M - 1$, moving to the next level cannot get any revenue since $r_{d_i+1} = r_M = 0$. Otherwise, level $d_i + 2$ exists when $d_i < M - 1$. There exist at least c_{d_i+2} links from level $d_i + 1$ to level $d_i + 2$. Before i moves to level $d_i + 1$, there exists at least one link from a node at level $d_i + 1$ to each node at level $d_i + 2$. i has at most c_{d_i+2} links to cloud nodes at level $d_i + 2$, so i can get at most $\frac{1}{2}$ of revenue from level $d_i + 1$. Since $r_{d_i} = \frac{r_{d_i+1}((c_{d_i}-1)c_{d_i+1}+1)}{2}$, we have $\frac{r_{d_i}}{(c_{d_i}-1)c_{d_i+1}+1} \geqslant \frac{r_{d_i+1}}{2}$. Therefore, cloud node i always receives at least as much revenue from level d_i as from level $d_i + 1$.

As a result, for a given transaction t, cloud node i cannot increase its revenue by disconnecting from other nodes, while other links are not changed. □

With the knowledge of the proof of Theorem 2, we explain Algorithm 2 line by line. Lines 1 and 2 initialize the variables. Lines 3 to 5 compute the outdegree of each cloud node in $\mathcal{V}$, which is also the number of links to the next level. Lines 6 to 9 compute the total number of

cloud nodes at each level, and the total number of links from these cloud nodes to the next level. Line 10 finds the maximum level M, and sets the multiplier r_{M-1} to 1. Level M cannot receive revenue since the cloud nodes do not forward the transaction to more cloud nodes. Cloud nodes at level $M-1$ only serve cloud nodes at level M. For convenience, the multiplier of the $M-1$ level r_{M-1} with the least income is set to 1. Lines 11 to 14 establish a relationship between the revenue of each two adjacent levels, that is, $r_n = r_{n+1}((c_n - 1)c_{n+1} + 1)/2$. The reason we set the multiplier is discussed in the proof of Theorem 2. S is the total of the multiplier for each level, which means each level d_i finally receives r_{d_i}/S of the total revenue w. Lines 15 to 17 compute the final revenue a_i to each node i. a_i consists two parts, $r_{d_i}w/S$ is the total revenue to level d_i, and p_i/g_{d_i} is the rate that i receives from the total revenue of level d_i.

The computational time complexity of this algorithm is $O(|\mathcal{V}| + |\mathcal{E}'|)$. Given $|\mathcal{E}'| < |\mathcal{E}|$, the total complexity of Algorithm 1 and 2 is $O(|\mathcal{V}| + |\mathcal{E}|)$.

In summary, our incentive allocation algorithm aligns the stated goals. The algorithm allocates revenue based on the contributions of cloud nodes when broadcasting transactions. Theorem 2 proves that cloud nodes cannot increase their revenue by disconnecting from other cloud nodes. The algorithm with linear computational time complexity is acceptable to most cloud nodes. It is worth mentioning that user nodes cannot obtain revenue, since user nodes do not contribute to transaction forwarding.

5.4 Incentive Allocation for Serving User Nodes

As described in Section 4, we only consider user nodes that have participated in relevant transactions within the last H blocks. Since user nodes are treated uniformly in terms of contribution, when allocating revenue to cloud nodes, we focus solely on the number of user nodes served by each cloud node, denoted as $\mathcal{U}_i$. Moreover, the incentive allocation method described previously is unaffected by the number of user nodes, as user node count does not influence the transaction broadcast process in the network. Therefore, we introduce additional allocation components w_0, w_1 to the original allocation w, such that $w = w_0 + w_1$, where w_0 is the allocation computed by Algorithms 1 and 2, and w_1 represents the reward related to servicing user nodes. In other words, the original algorithms correspond to the special case when $w_1 = 0$.

Algorithm 3 gives the accurate incentive allocation considering user nodes:

Algorithm 3 Incentive Allocation for Serving User Nodes

Input: $\mathcal{V}, w_1, \{\mathcal{U}_i | \forall i \in \mathcal{V}\}$
Output: $\mathcal{A}' = \{a'_i | i \in \mathcal{V}\}$
1: $Tot \leftarrow 0$
2: **for all** $i \in \mathcal{V}$ **do**
3: $\quad Tot \leftarrow Tot + |\mathcal{U}_i|$
4: **end for**
5: **for all** $i \in \mathcal{V}$ **do**
6: $\quad a'_i \leftarrow w_1|\mathcal{U}_i|/Tot$
7: **end for**

The idea of Algorithm 3 is straightforward. It distributes a total revenue of w_1 proportionally based on the number of user nodes served by each cloud node. Consequently, the final revenue received by each cloud node i is $a_i + a'_i$.

6 Blockchain Design

In this section, we discuss the blockchain design of the ITFC blockchain system. We propose a specialized structure to maintain cloud network topology and incentive allocations.

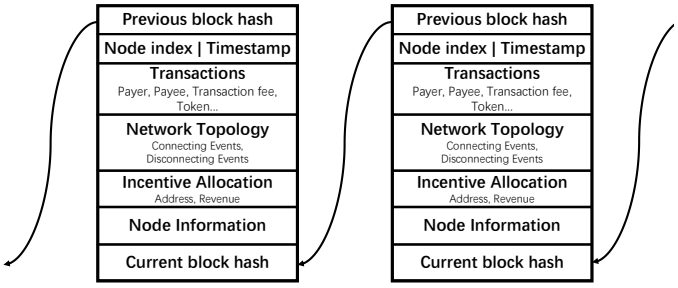


Fig. 1. An overview of the ITF block

6.1 ITFC Blockchain Structure

In addition to the traditional purposes of blockchain, our design must support the functions outlined in Sections 4 and 5. Fig. 1 provides an overview of the ITFC blockchain structure. ITFC records verification information and transactions, which is similar to traditional blockchains based on Nakamoto consensus [2]. Besides, ITFC has three additional fields: the network topology field, the incentive allocation field, and the node information field. The network topology and incentive allocations can be computed based on the data stored in the above fields, allowing all cloud nodes to reach a consensus. The node information field includes the necessary node information, including the cloud node registrations and active user nodes in the current block.

6.1.1 Network topology field

Recording the whole network information onto a single block is unacceptable. The network topology is indicated by accumulating all changes over time. The network topology field is necessary to track changes in network topology. The network topology field records the network topology information, including messages indicating connecting and disconnecting events. Blocks will record the connecting messages from both cloud nodes, and blocks will record the disconnecting message from either node of the link.

6.1.2 Incentive allocation field

The incentive allocation field records the final result of incentive allocation. To indicate the result of incentive allocation, the block records the address and revenue for cloud nodes in the block. The block generator computes the incentive allocation for all transactions in the current block and stores it in the block.

Every cloud node in the blockchain network has the motivation and ability to compute the network topology

and the incentive allocation. If the incentive allocation in the block is incorrect, the block will not be approved by other cloud nodes. Transactions and incentive allocation in unapproved blocks are not acknowledged by other cloud nodes. Therefore, block generators are motivated to correct the network topology field and the incentive allocation field.

6.1.3 Node information field

This field records the registration information of the newly added cloud nodes and active user nodes.

A newly added cloud node broadcasts a registration message to let the participants in the network know that it is capable of serving as a cloud node. The related message is recorded in the field. Other cloud nodes can communicate with the newly added cloud node and build links if possible.

As stated in Section 4, we consider only active user nodes whose activity satisfies $k > n - H$. These nodes are obtained by collecting all user nodes that appear in any transaction from the $n - H + 1$-th block to the current n-th block. The node information field records all user nodes appearing in the transactions of the current block for convenience, eliminating the need to traverse all transactions to obtain this information.

6.2 Network Topology Maintenance

With the support of the network topology field, we introduce how to maintain the network topology.

6.2.1 Network topology consensus in the blockchain

Cloud nodes in the network generate connecting and disconnecting messages to indicate continuous changes in the network topology. Ideally, all cloud nodes can reach a consensus on the network following the time order of all network changes. Practically, it is hard to strictly follow the time order, because time synchronization is hard among all cloud nodes. An unambiguous time system is needed for nodes to confirm the time order of different events. Even if an unambiguous time system exists, Inevitable network latency prevents all changes from being synchronized immediately across the network. There is no guarantee that all network changes can be tracked immediately and chronologically by all cloud nodes.

Thus, ITFC uses the index of the block as the signal for network topology consensus. Each block and its previous blocks record immutable network topology changes. These confirmed network topology changes construct an immutable network topology. More specifically, for a blockchain $\mathcal{B} = \{B_1, B_2, ..., B_n\}$ with the latest block B_n, once an index i where $i \in [1, n]$ is selected, there will exist a list of corresponding network topology changes which includes all changes from B_1 to B_i. In this case, all cloud nodes can reach the network topology consensus, which is a part of the blockchain consensus. To avoid a block generator selecting connecting and disconnecting events in the current block maliciously, topology changes in the current block have no impact on incentive allocations in the current block. Incentive allocations in block B_n apply the network topology accumulated by network topology changes from B_1 to B_{n-1}.

6.2.2 Storage consumption

The storage consumption of the network topology field is related to the number of connecting events and the storage consumption of each connecting event. In general, the number of connecting events does not exceed the number of transactions in the long term, because the purpose of connecting links in the network is to help transmit transactions. Establishing a link facilitates the transfer of multiple transactions between two nodes. If the number of connecting events is greater than the number of transactions, there must exist redundant links. Since connecting events require payments, nodes will not establish too many redundant links. Meanwhile, the number of disconnecting events cannot exceed the number of connecting events. A node will not propose disconnecting events unnecessarily for the potential future use of links.

A connecting event message only needs to include basic information, such as the addresses and signatures of both nodes, which consumes significantly fewer resources than a typical transaction. Similarly, a disconnecting event message contains comparable information and incurs a similarly low cost. In practice, the storage required for the network topology field is therefore much smaller than that needed for transactions.

Following the current Bitcoin network, there are approximately 22000 cloud nodes, each with at most 8 outbound connections, and a total of approximately 910000 blocks as of July 2025 [36], [38]. Assuming that the network information is uniformly distributed across all blocks, a new cloud node appears approximately once every 42 blocks, while a new link appears about once every 6 blocks. In practice, dynamic topological changes will produce substantially more data. However, due to the relative stability of cloud-based nodes, the magnitude of such changes is expected to be moderate. Under a conservative assumption that the volume of topological change information is up to 100 times greater than that of the final topology information, the resulting information can still be accommodated within the available block capacity.

6.3 Incentive Allocation Maintenance

6.3.1 Incentive allocation storage

An intuitive approach to distributing incentives is for the transaction payer to compute the allocation and create $O(|\mathcal{V}|)$ sub-transactions. However, this method introduces substantial overhead: each transaction would generate $O(|\mathcal{V}|)$ sub-transactions that must be broadcast across the entire blockchain network. Moreover, because the allocation details must be recorded on-chain, this approach would lead to explosive growth in blockchain size. Therefore, it is essential to design an efficient way to record incentive allocations.

As shown in Fig. 1, the incentive allocation field records three entries for each node that receives revenue:

- **Address**: The wallet address of the node.
- **Revenue**: The amount of revenue allocated to the node.

When a valid new block is generated, the system automatically distributes revenue according to the recorded allocation. Each transaction specifies its transaction fee, which

the payer deposits into a public reward pool. The miner then calculates the incentive allocation and updates the addresses of eligible nodes with their respective revenue shares. The incentive allocation field serves as an immutable reference for the distribution process.

6.4 Resource Consumption Analysis

The storage space required for the network topology field depends on the total number of cloud nodes. Each Bitcoin address is represented by a 20-byte hash, and the corresponding revenue is stored as an 8-byte real number, totaling 28 bytes per entry. For a network with 22000 cloud nodes, the incentive allocation requires approximately 616 KB of storage per block. If the address encoding method is properly compressed, the data size can be further reduced.

7 Attack Resistance

In this section, we analyze possible attacks to show that the ITFC system and the incentive allocation algorithm are secure against potential hazards.

7.1 Attacking Models

We assume that all nodes are selfish and rational. No node intends to tamper with the system without extra profits. Because the ITFC system inherits mining parts and mechanisms from Bitcoin [2], resistances of classical attacks like double-spending [41] are retained. We propose and analyze two potential attacks against the ITFC system.

7.1.1 Sybil attack

A Sybil attacker creates a large number of pseudonymous nodes and obtains extra profits through these nodes [14].

7.1.2 Active user node attack

The active user node attack is a targeted threat against the ITFC system. In this system, a cloud node earns higher revenue by serving more user nodes with related transactions in the most recent H blocks. Consequently, a cloud node might let user nodes continuously propose transactions to maintain their presence within the latest H blocks.

7.2 Safety Analysis

Without loss of generality, we assume there is one adversary that controls all adversarial nodes in the network. The primary target of the adversary is to increase the revenue from incentive allocations.

Essentially, the adversary attempts to gain extra profits by manipulating the graph formed by nodes in the activated set. They can create two types of fake identities: pseudonymous nodes and fake links, which are used to alter the network topology and influence incentive allocations. These fake identities are claimed by the adversary but are not maintained, so they cannot provide actual services. Otherwise, they would be legitimate identities, and the adversary could earn revenue from them. In theory, the adversary can generate an unlimited number of pseudonymous nodes, most of which initially have no links. Adding a node corresponds to creating links to it, while removing a node corresponds to deleting all its links. Therefore, node operations can be reduced to link operations. Moreover, as we prove in Section 5.3, the adversary cannot gain extra profits by removing links. Consequently, all adversarial operations can be effectively simplified to adding fake links.

After adding links, the adversary may gain extra profits in two scenarios. First, our incentive allocation algorithm assigns different total revenue to different levels. By altering shortest paths, the adversary changes node levels and can receive different revenue. Second, because the total revenue at each level is fixed, the adversary can increase its profits by concentrating adversarial nodes at specific levels, thereby capturing a larger share of the allocated revenue.

7.2.1 Changing shortest paths

The decisive factor of our incentive allocations algorithm is the shortest paths. To demonstrate the viability of possible attacks, we distinguish attacks by whether the shortest path changes.

To shorten the shortest paths, an adversary must declare fake links between cloud nodes whose original shortest path distance is at least 2. We first consider fake links connecting an honest cloud node and an adversarial cloud node. When an honest cloud node receives a transaction via any link, it can estimate the delivery time from all links based on publicly available topology information. Since fake links cannot deliver transactions within the expected time, honest cloud nodes can detect these fraudulent links and disconnect from the corresponding adversarial nodes.

Next, we consider the adversary adding fake links between adversarial cloud nodes. Generally, at least one honest cloud node's shortest path is improved by such a link involving an adversarial cloud node, meaning this honest cloud node is expected to receive information earlier through that link. However, since the link between adversarial cloud nodes is fake, the honest cloud node will not receive transactions within the expected time frame. Consequently, the honest cloud node will disconnect from links that fail to forward transactions promptly, even if those links are genuine. As a result, the adversary forfeits the revenue from serving this honest cloud node.

As a special case, we consider that none of the shortest paths between any two honest cloud nodes is improved. This implies that the adversary cannot gain additional profits from transactions originating from honest nodes and can only earn extra revenue from transactions involving adversarial nodes. It is important to note that most transaction fees are paid to honest nodes, as they typically possess the majority of the computing power required to generate blocks. Therefore, the adversary cannot leverage honest node transactions to increase its profits beyond the standard incentive allocation.

7.2.2 Maintain active user nodes

To maintain active user nodes, the adversary generates transactions between its own user nodes, enabling the associated cloud nodes to earn revenue. However, for block generators to validate these transactions, the adversary must incur significant transaction fees. The majority of transaction fees are awarded to honest nodes. Consequently, the

adversary must regularly create transactions and pay corresponding fees to sustain user node activity. Our evaluations in Section 8 demonstrate that this approach does not yield additional profits for the adversary.

8 Performance Evaluation

In this section, we provide the results of the performance of the ITFC blockchain. We show the incentive allocation performance. We also conduct attacks to test the security of the ITFC blockchain. We write code to simulate all nodes, and they operate the same blockchain.

8.1 Distribution of Incentive Allocation

To show the contribution of cloud nodes in forwarding processes, we define **sufficient forwarding** in Section 5.3. Theoretically, cloud nodes that make more sufficient forwarding contribute more during the forwarding processes and may receive more revenue. Therefore, we compare the sufficient forwarding times of cloud nodes and the revenue they receive.

We generate the test network by using algorithms in [42]. This algorithm generates a realistic model, incorporating hierarchy and redundancy. The number of links of each cloud node varies from 4 to 60. The range of the number of links is sufficient to show the relationship between the number of links and the revenue of cloud nodes, and there are enough samples for each number of links in the generated graph.

The test network consists of 10000 cloud nodes. Each cloud node serves one user node, and each user node broadcasts a transaction once to be active. We compute the final revenue and the number of sufficient forwarding times for each cloud node. The activated set includes all cloud nodes initially, thus all cloud nodes can receive transaction fees for relay nodes. We assume that all honest nodes broadcast transactions with the same amount of transaction fees f_0, which is called the **standard transaction fee**. Assume that a node costs f transaction fees and receives u revenue. f includes transaction fees for broadcasting transactions, while u includes revenue for relay nodes and block generators. We define the **profit rate** as $(u - f)/f_0$.

We assume that each node has the same probability of becoming a block generator, and all cloud nodes will receive the same proportion of transaction fees for block generators. To show the impact of the transaction fees for relay cloud nodes, we set the transaction fees for relay nodes to 50%.

Fig. 2(a) and (b) show the distribution of the profit rate and the number of sufficient forwarding times. The profit rate increases as the number of links grows. We further analyze the data to explore the relationship among profit rate, sufficient forwarding times, and the number of links. Of particular interest is the **average unit profit rate**, defined as the average profit rate per sufficient forwarding. Fig. 2(c) illustrates that the average unit profit rate rises with the number of links. Note that nodes with negative profit rates transfer revenue to nodes with positive profit rates in the network. Most nodes with fewer than 22 links have a negative average unit profit rate, whereas nodes with more than 22 links exhibit a positive average unit profit rate. This indicates that nodes with more links gain higher revenue per sufficient forwarding. The line fluctuates when the number of nodes exceeds 50 due to insufficient sample sizes for high-link nodes.

We further examine the relationship between the number of links and the average profit rate per sufficient forwarding by computing the average unit profit rate normalized by the number of links. The dotted line in Fig. 2(c) represents this metric. It increases steadily for a small number of links and stabilizes as the number of links reaches a threshold. Since the average unit profit rate accumulates contributions from all links, we conclude that a node's revenue grows approximately linearly with the number of links.

In summary, the incentive allocation algorithm distributes revenue according to nodes' contributions, encouraging nodes to enhance system connectivity by establishing links that support sufficient forwarding.

As a result, the incentive allocation algorithm distributes revenue to nodes based on their contributions. It encourages nodes to improve the connectivity of the system by setting up links that support sufficient forwarding.

8.2 Sybil Attack

To test the resistance to the Sybil attack we mentioned in Section 7.1, we generate the test network by Watts-Strogatz model [43]. This model generates networks that reveal the properties of some real communication networks. Compared with the algorithm in [42], the distribution of the link number of each node in the graph generated by this algorithm is more concentrated, which is clearer for observing the impact of the attack.

We now analyze the cost and profit of the Sybil attack for simulation design. The cost of the Sybil attack is for pseudonymous nodes created by the adversary to join the activated set. Each pseudonymous node needs to pay a transaction fee to join the activated set. Since the creation of a pseudonymous node cannot improve the computing power of the adversary, the adversary cannot receive more revenue for block generators. Therefore, there are two possibilities for increasing profit, including reducing costs and creating pseudonymous nodes to obtain more revenue for relay nodes. Based on the composition of cost and profit, the simulations will compare the profit of the adversary when it creates different numbers of pseudonymous nodes and pays different amounts of transaction fees for pseudonymous nodes.

The test network consists of 1000 cloud nodes. We randomly select one cloud node as the adversary node, which creates multiple pseudonymous cloud nodes. Together, the adversary and its pseudonymous nodes form a complete graph. Each node broadcasts one transaction, and each pseudonymous node also broadcasts one transaction to join the activated set. Initially, the activated set includes all nodes, allowing every node to receive transaction fees as relay nodes. We assume all honest nodes broadcast transactions with the standard fee f_0. The adversary pays f for each of its pseudonymous nodes and the adversary node itself. When the adversary pays yf_0 for each of x pseudonymous nodes, the total expenditure is $f = xyf_0$. All nodes are assumed to have equal computing power, giving each node the same probability of being selected as a

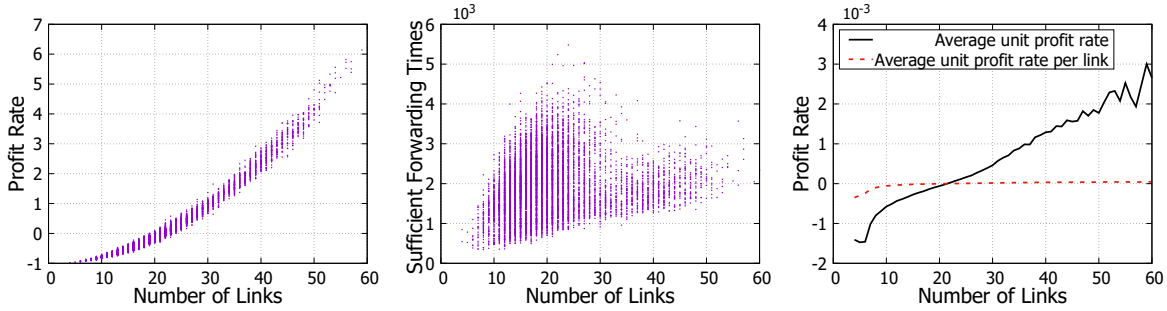


Fig. 2. The simulation result of the generated network with 10000 cloud nodes. (a) shows the distribution of the number of links and the profit rate. (b) shows the distribution of the number of links and the number of sufficient forwarding times. (c) shows the average unit profit rate and the average unit profit rate per link for each number of links.

block generator. Pseudonymous nodes cannot serve as block generators because the adversary's computing power is limited. The adversary's profit u is the total revenue received by the adversary node and all pseudonymous nodes. We evaluate the total profit rate as $(u - f)/f_0$, where a positive value indicates that the adversary gains extra profit. To demonstrate the resistance of our algorithm to Sybil attacks, we consider the worst-case scenario where the adversary maximizes revenue. In this setup, block generators receive 50% of transaction fees, while relay nodes receive the remaining 50%.

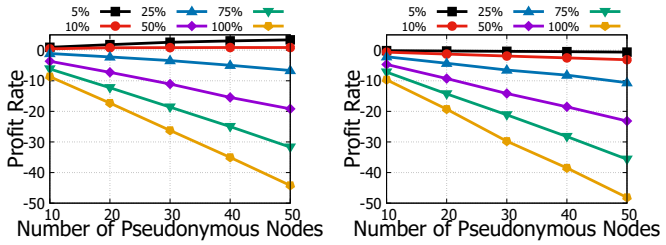


Fig. 3. The simulation result of the Sybil attack. Different lines indicate the adversary pays different percentages of the standard transaction fee to join the activated set. The mean degree of each node is 10 in (a), 50 in (b).

Fig. 3 shows that the profit rate grows linearly with the number of pseudonymous nodes. When the slope is positive, more pseudonymous nodes obtain more profits. Otherwise, more pseudonymous nodes suffer more losses. In Fig. 3(a), the adverse node can obtain extra profits if pseudonymous nodes broadcast transactions with no more than 10% of standard transaction fees. The adversary cannot obtain extra profits in Fig. 3(b). The difference between (a) and (b) is the mean degree of each node. We can infer that improving connectivity can improve the safety of our system.

We conclude that our system can resist the Sybil attack. There are two reasons that make the Sybil attack ineffective. First, the adversary only obtains extra profits when their transactions with low transaction fees can be accepted. However, block generators are honest most of the time. They always choose transactions with higher transaction fees for more revenue. It avoids pseudonymous nodes joining the activated set at a low cost. Second, the adversary cannot obtain the revenue for block generators. The adversary can only obtain the revenue for relay nodes. As a result, the adversary cannot obtain extra profits through the Sybil attack, thus our system can resist the Sybil attack.

8.3 Active User Node Attack

We design a simulation to evaluate the performance of the activated set attack. Building on the simulations in Section 8.1, we incorporate user nodes to test the effectiveness of the active user node attack. The test network configuration follows the same setting as in Section 8.1.

In the simulation, a cloud node with 50 links is designated as the adversarial node. To continuously obtain transaction fees, the adversary keeps its associated user nodes active by repeatedly proposing transactions. Considering that each Bitcoin block contains approximately 3000 transactions [44], the simulation generates 1000 transactions per block from distinct user nodes, uniformly distributed among all cloud nodes. With $H = 10$, only cloud nodes serving active user nodes within the latest 10 blocks are eligible for revenue. Consequently, the adversary's user nodes must generate at least one transaction every 10 blocks to maintain their activity. For simplicity, the simulation scales proportionally, using 10000 transactions per block and $H = 1$, with the adversary maintaining multiple user nodes and submitting one transaction per user node per block.

We assume that all honest user nodes broadcast transactions with the standard transaction fee f_0, while the adversary pays a fee f for transactions submitted by all its user nodes. The adversary's profit u is defined as the total revenue received from relaying and serving its user nodes. The profit rate is computed as $(u - f)/f_0$. Transaction fees are divided equally, with 50% allocated to block generators and 50% to relay nodes. Ignoring mining randomness, we assume that the revenue allocated to block generators is uniformly distributed across all cloud nodes.

Fig. 4 shows that the profit rate decreases linearly as the number of blocks increases in all cases. In the basic setting, with $w_0 = 1$ and each user node paying f_0, Fig. 4(a) illustrates that the profit rate declines linearly with both the number of blocks and the number of adversary-controlled user nodes. In Fig. 4(b), where $w_0 = 0.5$ indicating that half of the fees allocated to relay cloud nodes are distributed to cloud nodes serving user nodes, the adversary cannot gain additional profit by increasing the number of user nodes. The profit rates are even lower compared to Fig. 4(a). This

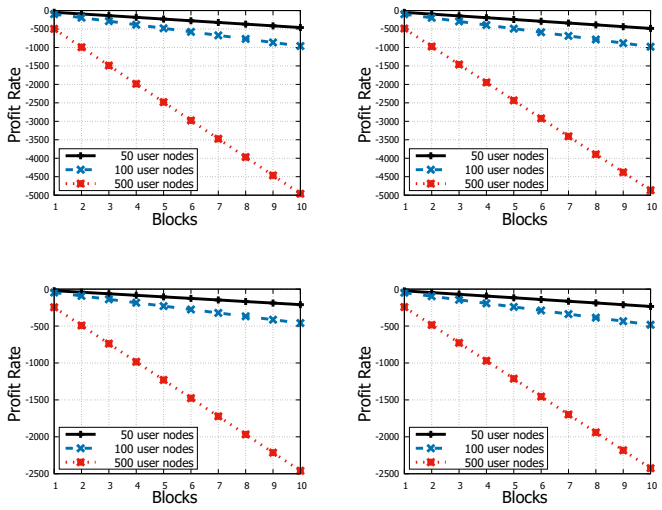


Fig. 4. Simulation results of the active user node attack for different numbers of active user nodes controlled by the adversary cloud node. (a) shows the results with $w_0 = 1$ and each user node paying f_0. (b) shows the results with $w_0 = 0.5$ and each user node paying f_0. (c) shows the results with $w_0 = 1$ and each user node paying $0.5f_0$. (d) shows the results with $w_0 = 0.5$ and each user node paying $0.5f_0$.

occurs because when the adversary adds a new user node to propose a transaction, Algorithm 2 assigns higher fees to the adversary node as it starts broadcasting at a low level. However, the increase in fees for serving user nodes does not compensate for the reduction in fees allocated to relay cloud nodes, resulting in a decrease in profit for the adversary.

We adjust the cost of proposing a transaction for each adversary-controlled user node, reducing it to $0.5f_0$ in Fig. 4(c). As a result, the overall profit rate becomes approximately half of the rate under the same conditions in Fig. 4(a). This demonstrates that the change in profit rate is due to the reduced transaction cost, and the adversary still cannot gain any profit. Fig. 4(d) shows the results when $w_0 = 0.5$. Similar to Fig. 4(c), the profit rates are roughly half of the corresponding values in Fig. 4(b). These results confirm that our system effectively resists the active user node attack under different w_0 settings and reduced transaction fee costs for user nodes.

Theoretically, the adversary could gain extra profits if the transaction fees for user nodes are extremely low. However, such transactions can be easily detected by other cloud nodes, and their priority for approval is low. Therefore, we conclude that our system effectively resists the activated set attack.

9 Conclusion

In this paper, we have proposed the ITFC blockchain system, focusing on secure incentive allocation in blockchain networks. We have introduced a mechanism to maintain the network topology, encompassing both cloud and user nodes. We have developed an efficient incentive allocation algorithm to reward cloud nodes based on their contributions to transaction forwarding and user service. We have analyzed the security of ITFC and the allocation algorithm against several types of attacks. We have conducted extensive simulations that demonstrate our system fairly rewards cloud nodes while effectively resisting various adversarial behaviors.

Acknowledgement

This work is supported in part by US National Science Foundation under grant number 2230620.

References

[1] D.-J. Deng, K.-C. Chen, and R.-S. Cheng, "Ieee 802.11 ax: Next generation wireless local area networks," in *10Th international conference on heterogeneous networking for quality, reliability, security and robustness*. IEEE, 2014, pp. 77–82.

[2] S. Nakamoto *et al.*, "Bitcoin: A peer-to-peer electronic cash system," 2008.

[3] M. Babaioff, S. Dobzinski, S. Oren, and A. Zohar, "On bitcoin and red balloons," in *Proceedings of the 13th ACM conference on electronic commerce*, 2012, pp. 56–73.

[4] I. Eyal and E. G. Sirer, "Majority is not enough: Bitcoin mining is vulnerable," in *International Conference on Financial Cryptography and Data Security*. Springer, 2014, pp. 436–454.

[5] R. Pass and E. Shi, "Fruitchains: A fair blockchain," in *Proceedings of the ACM Symposium on Principles of Distributed Computing*. ACM, 2017, pp. 315–324.

[6] Y. Amoussou-Guenou, A. Del Pozzo, M. Potop-Butucaru, and S. Tucci-Piergiovanni, "Correctness and fairness of tendermint-core blockchains," *arXiv preprint arXiv:1805.08429*, 2018.

[7] M. Feldman, K. Lai, I. Stoica, and J. Chuang, "Robust incentive techniques for peer-to-peer networks," in *Proceedings of the 5th ACM conference on Electronic commerce*. ACM, 2004, pp. 102–111.

[8] X. Yan, F. Ye, Y. Yang, and X. Deng, "An autonomous compensation game to facilitate peer data exchange in crowdsensing," in *2017 IEEE/ACM 25th International Symposium on Quality of Service (IWQoS)*. IEEE, 2017, pp. 1–6.

[9] Y. He, H. Li, X. Cheng, Y. Liu, C. Yang, and L. Sun, "A blockchain based truthful incentive mechanism for distributed p2p applications," *IEEE Access*, vol. 6, pp. 27 324–27 335, 2018.

[10] L. Buttyan, L. Dora, M. Felegyhazi, and I. Vajda, "Barter-based cooperation in delay-tolerant personal wireless networks," in *2007 IEEE International Symposium on a World of Wireless, Mobile and Multimedia Networks*. IEEE, 2007, pp. 1–6.

[11] K. Zhao, S. Tang, B. Zhao, and Y. Wu, "Dynamic and privacy-preserving reputation management for blockchain-based mobile crowdsensing," *IEEE Access*, vol. 7, pp. 74 694–74 710, 2019.

[12] D. Liu, A. Alahmadi, J. Ni, X. Lin *et al.*, "Anonymous reputation system for iiot-enabled retail marketing atop pos blockchain," *IEEE Transactions on Industrial Informatics*, 2019.

[13] M. Feldman, C. Papadimitriou, J. Chuang, and I. Stoica, "Free-riding and whitewashing in peer-to-peer systems," *IEEE Journal on selected areas in communications*, vol. 24, no. 5, pp. 1010–1019, 2006.

[14] Z. Trifa and M. Khemakhem, "Sybil nodes as a mitigation strategy against sybil attack," *Procedia Computer Science*, vol. 32, pp. 1135–1140, 2014.

[15] F. Franzoni, X. Salleras, and V. Daza, "Atom: Active topology monitoring for the bitcoin peer-to-peer network," *Peer-to-Peer Networking and Applications*, vol. 15, no. 1, pp. 408–425, 2022.

[16] M. Essaid, C. Lee, and H. Ju, "Characterizing the bitcoin network topology with node-probe," *International Journal of Network Management*, vol. 33, no. 6, p. e2230, 2023.

[17] Z. Zhao, J. Wang, M. Yang, and H. Wang, "An efficient bitcoin network topology discovery algorithm for dynamic display," *Blockchain: Research and Applications*, p. 100260, 2025.

[18] J. Poon and T. Dryja, "The bitcoin lightning network: Scalable off-chain instant payments," 2016.

[19] V. Sivaraman, S. B. Venkatakrishnan, K. Ruan, P. Negi, L. Yang, R. Mittal, G. Fanti, and M. Alizadeh, "High throughput cryptocurrency routing in payment channel networks," in *17th {USENIX} Symposium on Networked Systems Design and Implementation ({NSDI} 20)*, 2020, pp. 777–796.

[20] Z. Avarikioti, L. Heimbach, Y. Wang, and R. Wattenhofer, "Ride the lightning: The game theory of payment channels," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 264–283.

[21] O. Ersoy, S. Roos, and Z. Erkin, "How to profit from payments channels," in *International Conference on Financial Cryptography and Data Security*. Springer, 2020, pp. 284–303.

[22] L. Luu, V. Narayanan, C. Zheng, K. Baweja, S. Gilbert, and P. Saxena, "A secure sharding protocol for open blockchains," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security*, 2016, pp. 17–30.

[23] M. Zhang, J. Li, Z. Chen, H. Chen, and X. Deng, "Cycledger: A scalable and secure parallel protocol for distributed ledger via sharding," *arXiv preprint arXiv:2001.06778*, 2020.

[24] C. Huang, Z. Wang, H. Chen, Q. Hu, Q. Zhang, W. Wang, and X. Guan, "Repchain: A reputation based secure, fast and high incentive blockchain system via sharding," *IEEE Internet of Things Journal*, 2020.

[25] Q. Wang, X. Zhu, Y. Ni, L. Gu, and H. Zhu, "Blockchain for the iot and industrial iot: A review," *Internet of Things*, vol. 10, p. 100081, 2020.

[26] Z. Xiong, Y. Zhang, D. Niyato, P. Wang, and Z. Han, "When mobile blockchain meets edge computing," *IEEE Communications Magazine*, vol. 56, no. 8, pp. 33–39, 2018.

[27] X. Ding, J. Guo, D. Li, and W. Wu, "An incentive mechanism for building a secure blockchain-based internet of things," *IEEE Transactions on Network Science and Engineering*, 2020.

[28] Y. Huang, Y. Zeng, F. Ye, and Y. Yang, "Incentive assignment in pow and pos hybrid blockchain in pervasive edge environments," in *2020 IEEE/ACM 28th International Symposium on Quality of Service (IWQoS)*. IEEE, 2020, pp. 1–10.

[29] I. Abraham, D. Malkhi, K. Nayak, L. Ren, and A. Spiegelman, "Solidus: An incentive-compatible cryptocurrency based on permissionless byzantine consensus," *CoRR, abs/1612.02916*, 2016.

[30] O. Ersoy, Z. Ren, Z. Erkin, and R. L. Lagendijk, "Transaction propagation on permissionless blockchains: incentive and routing mechanisms," in *2018 Crypto Valley Conference on Blockchain Technology (CVCBT)*. IEEE, 2018, pp. 20–30.

[31] O. Ersoy, E. Zekeriya, and R. L. Lagendijk, "Tulip: A fully incentive compatible blockchain framework amortizing redundant communication," in *2019 IEEE European Symposium on Security and Privacy Workshops (EuroS&PW)*. IEEE, 2019, pp. 396–405.

[32] L. Li, J. Liu, L. Cheng, S. Qiu, W. Wang, X. Zhang, and Z. Zhang, "Creditcoin: A privacy-preserving blockchain-based incentive announcement network for communications of smart vehicles," *IEEE Transactions on Intelligent Transportation Systems*, vol. 19, no. 7, pp. 2204–2220, 2018.

[33] X. Wang, Y. Chen, and Q. Zhang, "Incentivizing cooperative relay in utxo-based blockchain network," *Computer Networks*, vol. 185, p. 107631, 2021.

[34] Statista, "Bitcoin blockchain size 2009-2025," "https://www.statista.com/statistics/647523/worldwide-bitcoin-blockchain-size/", 2025.

[35] MiningPoolsStats, "Bitcoin (btc) sha-256 — mining pools," "https://miningpoolstats.stream/bitcoin", 2025.

[36] Bitcoin.org, "P2p network guide," "https://developer.bitcoin.org/devguide/p2p_network.html", 2025.

[37] Bitinfocharts, "Bitcoin active addresses chart," "https://bitinfocharts.com/comparison/bitcoin-activeaddresses.html#", 2025.

[38] Bitnodes, "Global bitcoin nodes distribution," "https://bitnodes.io/", 2025.

[39] T. Chen, Y. Zhu, Z. Li, J. Chen, X. Li, X. Luo, X. Lin, and X. Zhange, "Understanding ethereum via graph analysis," in *IEEE INFOCOM 2018-IEEE Conference on Computer Communications*. IEEE, 2018, pp. 1484–1492.

[40] S. Delgado-Segura, S. Bakshi, C. Pérez-Solà, J. Litton, A. Pachulski, A. Miller, and B. Bhattacharjee, "Txprobe: Discovering bitcoin's network topology using orphan transactions," in *International Conference on Financial Cryptography and Data Security*. Springer, 2019, pp. 550–566.

[41] U. W. Chohan, "The double spending problem and cryptocurrencies," *Available at SSRN 3090174*, 2017.

[42] M. B. Doar, "A better model for generating test networks," in *Proceedings of GLOBECOM'96. 1996 IEEE Global Telecommunications Conference*. IEEE, 1996, pp. 86–93.

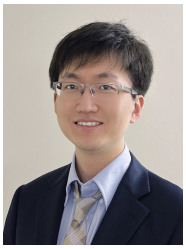
[43] D. J. Watts and S. H. Strogatz, "Collective dynamics of 'small-world' networks," *nature*, vol. 393, no. 6684, p. 440, 1998.

[44] Ycharts, "Bitcoin average transactions per block," "https://ycharts.com/indicators/bitcoin_average_transactions_per_block", 2025.

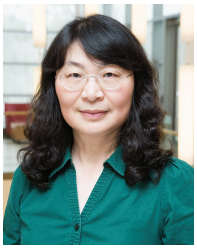
Jiarui Zhang received the B.Eng. degree in computer science and technology from Shanghai Jiao Tong University in 2017. He is currently working towards the Ph.D. degree in computer engineering at Stony Brook University. His research interests include blockchain, mobile edge computing, quantum networking and computing.

Yaodong Huang (Member, IEEE) received the Ph.D. degree in computer engineering from Stony Brook University, New York, NY, USA. He is currently an Assistant Professor with the College of Computer Science and Software Engineering, Shenzhen University. His research interests include mobile and edge computing, with focus on communication protocols, cooperative caching, security, privacy, energy efficiency, virtualization, and novel network architectures in edge computing environments.

Yiming Zeng is an assistant professor in the School of Computing at Binghamton University (SUNY at Binghamton). He earned his Ph.D. from Stony Brook University, New York, USA, and his B.Eng. degree from Shanghai Jiao Tong University, Shanghai, China. His research currently centers on quantum networking and quantum computing.



Xiaojun Shang received his B. Eng. degree in Information Science and Electronic Engineering from Zhejiang University and his M.S. degree in Electronic Engineering from Columbia University. He received his Ph.D. degree in Computer Engineering from Stony Brook University. Dr. Shang is currently an assistant professor in the Department of Computer Science and Engineering at the University of Texas, Arlington. His research interests include the areas of mobile edge computing, software defined networking, and quantum computing.



Yuanyuan Yang received the B.Eng. and M.S. degrees in computer science and engineering from Tsinghua University, Beijing, China, and the M.S.E. and Ph.D. degrees in computer science from Johns Hopkins University, Baltimore, Maryland. She is currently a SUNY Distinguished Professor in the Department of Electrical & Computer Engineering and Department of Computer Science at Stony Brook University, New York, which she joined in 1999. From 2018-2022, she served as a program director in the National Science Foundation's Directorate of Computer and Information Science and Engineering. She directed the core computer architecture program and was on the management team of several cross-cutting programs. She has authored or coauthored more than 550 papers in major journals and refereed conference proceedings and holds seven US patents in her areas of research which include quantum computing, edge/cloud computing, and mobile computing. She has served as the Editor-in-Chief for IEEE Transactions on Cloud Computing, and the Associate Editor-in-Chief and Associated Editor for IEEE Transactions on Computers. She is currently an Associate Editor for IEEE Transactions on Parallel and Distributed Systems and IEEE Transactions on Sustainable Computing. She was also the General Chair, Program Chair, or Vice Chair for several major conferences and a Program Committee Member for numerous conferences. She is a Fellow of National Academy of Inventors (NAI) and a Fellow of IEEE.